

Python Interfaces und Beispiele

Das jupyterturtle-Modul

Das jupyterturtle-Modul bietet eine einfache Möglichkeit, grafische Zeichnungen durch Programmierung zu erstellen. Es basiert auf dem Konzept einer "Schildkröte", die sich auf einer Zeichenfläche bewegt und dabei eine Spur hinterlässt.

Import und grundlegende Funktionen

```
# Vollständiger Import
import jupyterturtle

# Oder Import einzelner Funktionen
from jupyterturtle import make_turtle, forward, left, right, penup, pendown
```

Wichtige Befehle für die Schildkröte

Befehl	Beschreibung
<code>make_turtle()</code>	Erstellt eine Zeichenfläche und eine Schildkröte
<code>forward(length)</code>	Bewegt die Schildkröte vorwärts um <code>length</code> Einheiten
<code>left(angle)</code>	Dreht die Schildkröte um <code>angle</code> Grad nach links
<code>right(angle)</code>	Dreht die Schildkröte um <code>angle</code> Grad nach rechts
<code>penup()</code>	Hebt den Stift an (bewegt ohne zu zeichnen)
<code>pendown()</code>	Senkt den Stift ab (zeichnet beim Bewegen)

Beispiel: Ein einfaches Quadrat zeichnen

```
make_turtle()
forward(50)
left(90)
forward(50)
left(90)
forward(50)
left(90)
forward(50)
left(90)
```

Beispiel: Ein Quadrat mit einer Schleife zeichnen

```
make_turtle()
for i in range(4):
    forward(50)
    left(90)
```

Verkapselung und Verallgemeinerung

Verkapselung

Verkapselung bedeutet, eine Folge von Anweisungen in eine Funktion einzukapseln. Dies macht den Code übersichtlicher und wiederverwendbar.

```
def square():
    for i in range(4):
        forward(50)
        left(90)

# Aufruf der Funktion
make_turtle()
square()
```

Verallgemeinerung

Verallgemeinerung bedeutet, konkrete Werte durch Parameter zu ersetzen, um die Funktion flexibler zu machen.

```
def square(length):
    for i in range(4):
        forward(length)
        left(90)

# Aufruf mit verschiedenen Größen
make_turtle()
square(30) # Kleines Quadrat
square(60) # Größeres Quadrat
```

Weitere Verallgemeinerung: Ein Polygon zeichnen

```
def polygon(n, length):
    angle = 360 / n
    for i in range(n):
        forward(length)
        left(angle)

# Beispielaufrufe
```

```
make_turtle()
polygon(3, 50) # Dreieck
polygon(5, 30) # Fünfeck
polygon(8, 20) # Achteck
```

Schlüsselwortargumente

Schlüsselwortargumente machen den Code lesbarer, besonders bei Funktionen mit mehreren Parametern.

```
# Ohne Schlüsselwortargumente
polygon(7, 30)

# Mit Schlüsselwortargumenten
polygon(n=7, length=30) # Deutlicher, wofür die Werte stehen
```

Kreis- und Bogenzeichnung

Kreis zeichnen durch Näherung mit einem Polygon

```
import math

def circle(radius):
    circumference = 2 * math.pi * radius
    n = 30 # Anzahl der Segmente
    length = circumference / n
    polygon(n, length)
```

Refaktorisierung mit polyline

```
def polyline(n, length, angle):
    """Zeichnet n Liniensegmente mit gegebener Länge und Winkel.

    n: Anzahl der Segmente (Integer)
    length: Länge jedes Segments
    angle: Winkel zwischen den Segmenten (in Grad)
    """
    for i in range(n):
        forward(length)
        left(angle)
```

Polygon mit polyline implementieren

```
def polygon(n, length):
    """Zeichnet ein regelmäßiges n-Eck mit Seitenlänge length."""
```

```
angle = 360.0 / n
polyline(n, length, angle)
```

Bogen zeichnen

```
def arc(radius, angle):
    """Zeichnet einen Bogen mit gegebenem Radius und Winkel."""
    arc_length = 2 * math.pi * radius * angle / 360
    n = 30 # Anzahl der Segmente
    length = arc_length / n
    step_angle = angle / n
    polyline(n, length, step_angle)
```

Kreis mit arc implementieren

```
def circle(radius):
    """Zeichnet einen Kreis mit gegebenem Radius."""
    arc(radius, 360)
```

Funktionsdesign und Interface

Interface vs. Implementierung

- **Interface:** Beschreibt, wie die Funktion verwendet wird (Name, Parameter, Aufgabe)
- **Implementierung:** Beschreibt, wie die Funktion intern arbeitet

Beispiel für unterschiedliche Implementierungen mit gleichem Interface

```
# Erste Implementierung von circle
def circle(radius):
    circumference = 2 * math.pi * radius
    n = 30
    length = circumference / n
    polygon(n, length)

# Zweite Implementierung (nach Refaktorisierung)
def circle(radius):
    arc(radius, 360)
```

Beide Funktionen haben das gleiche Interface (Name und Parameter), aber unterschiedliche Implementierungen.

Docstrings

Docstrings sind Zeichenketten am Anfang einer Funktion, die das Interface dokumentieren.

```
def polyline(n, length, angle):  
    """Zeichnet Liniensegmente mit gegebener Länge und Winkel.  
  
    n: Anzahl der Liniensegmente (ganzzahlig)  
    length: Länge der Liniensegmente  
    angle: Winkel zwischen den Segmenten (in Grad)  
    """  
    for i in range(n):  
        forward(length)  
        left(angle)
```

Konventionen für Docstrings

- Werden mit dreifachen doppelten Anführungszeichen umgeben ("""
- Enthalten eine kurze Beschreibung der Funktion
- Erklären die Parameter und deren Typ
- Beschreiben ggf. den Rückgabewert und wichtige Hinweise

Prä- und Postkonditionen

- **Präkonditionen:** Bedingungen, die vor der Ausführung der Funktion erfüllt sein müssen
 - Beispiel: `n` muss ein Integer sein, `length` eine positive Zahl
 - Die Verantwortung liegt beim Aufrufer
- **Postkonditionen:** Bedingungen, die nach der Ausführung der Funktion gelten
 - Beispiel: Eine bestimmte Form wurde gezeichnet, die Schildkröte hat ihre Position geändert
 - Die Verantwortung liegt bei der Funktion selbst

Entwicklungsplan: Verkapselung und Verallgemeinerung

1. Beginnen Sie mit einem kleinen, funktionierenden Programm ohne Funktionen
2. Identifizieren Sie zusammengehörige Codeblöcke und kapseln Sie diese in Funktionen ein
3. Verallgemeinern Sie die Funktionen durch Hinzufügen geeigneter Parameter
4. Wiederholen Sie die Schritte 1-3, bis Sie einen Satz fehlerfreier Funktionen haben
5. Verbessern Sie das Programm durch Refaktorisierung

Refaktorisierung

Refaktorisierung bedeutet, funktionierenden Code umzugestalten, um ihn zu verbessern, ohne sein Verhalten zu ändern.

Beispiel für Refaktorisierung

```
# Vor der Refaktorisierung: Ähnlicher Code in mehreren Funktionen  
  
def draw_triangle(side):
```

```

    for i in range(3):
        forward(side)
        left(120)

def draw_square(side):
    for i in range(4):
        forward(side)
        left(90)

# Nach der Refaktorisierung: Gemeinsame Struktur extrahiert

def polygon(n, side):
    angle = 360 / n
    for i in range(n):
        forward(side)
        left(angle)

def draw_triangle(side):
    polygon(3, side)

def draw_square(side):
    polygon(4, side)

```

Praktische Hilfsfunktionen

Bewegung ohne Zeichnen (Jump)

```

def jump(length):
    """Bewegt die Schildkröte vorwärts, ohne eine Spur zu hinterlassen.

    Postbedingung: Stift bleibt unten.
    """
    penup()
    forward(length)
    pendown()

```

Nützliche Tipps

1. **Beschleunigung der Zeichnung:** `make_turtle(delay=0.02)` setzt eine kürzere Verzögerung zwischen den Zeichenschritten
2. **Verschachtelte Funktionsaufrufe:** Komplexe Formen können durch Kombination einfacherer Funktionen erstellt werden
3. **Wiederholung von Formen:** Verwenden Sie Schleifen, um Formen zu wiederholen oder Muster zu erzeugen
4. **Nachvollziehbarkeit:** Stack-Diagramme helfen, den Ablauf verschachtelter Funktionsaufrufe zu verstehen

Stack-Diagramm für verschachtelte Funktionsaufrufe

```
circle  
radius = 30
```

↓

```
arc  
radius = 30  
angle = 360
```

↓

```
polyline  
n = 60  
length = 3.04  
angle = 5.8
```

Glossar

Begriff	Bedeutung
Zeichenfläche (Canvas)	Ein Bereich auf dem Bildschirm für grafische Darstellungen
Verkapselung	Prozess, eine Anweisungsfolge in eine Funktion zu verwandeln
Verallgemeinerung	Ersetzung spezifischer Werte durch Parameter
Schlüsselwortargument	Argument, das den Namen des Parameters enthält
Refaktorisierung	Umgestalten von Code bei gleichbleibendem Verhalten
Entwicklungsplan	Systematische Vorgehensweise zum Schreiben von Programmen
Docstring	Dokumentationstext am Anfang einer Funktion
Multiline-String	String, der sich über mehrere Zeilen erstreckt
Präkondition	Bedingung, die vor der Funktionsausführung erfüllt sein muss
Postkondition	Bedingung, die nach der Funktionsausführung gelten muss
Interface	Beschreibung, wie eine Funktion verwendet wird
Implementierung	Details, wie eine Funktion intern arbeitet